

VERIQO Qualification Architecture

Round: G Date: 2026-09-04 Branch: `claude/l99-gap-coverage-nv70zy` Input: `Another_findings.docx`

0. The change of question

This round is not a gap-closing round, and saying so first matters, because the review's central instruction was to stop treating it as one:

Kita sekarang harus berhenti berpikir: *"How do we close all gaps?"* dan mulai berpikir: *"What evidence is required to legitimately promote each control to the next assurance level?"*

Twenty-two ASSURANCE_GAP rows are not a backlog. Most of them cannot be closed by writing code at all — they are waiting on somebody who is not VERIQO. Treating them as engineering tasks produces exactly the pressure that closes gaps dishonestly, and produces a roadmap that appears to converge and never does.

So the deliverable of this round is `pkg/assurance/plan`: every open control with a **proof obligation, test method, evidence artefact, pass/fail criteria and named validator**, and a rule in code that VERIQO may not be the validator of its own promotion.

Along the way, three of the new checks found real defects. Those are in §6, and one of them is a genuine Article 28 violation that had been sitting in the trust layer.

1. Four conditions, not one

The review drew a distinction that turns out to be the sharpest thing in the document:

test yang mendeteksi bug tertentu $\neq$ test yang membuktikan seluruh kelas bug sudah tertutup

`TestNoDocumentMisquotesTheRuntimeLedger` proved **Correctness** and was being read as proving all four. It now proves one condition, and three sibling tests prove the others, each able to fail on its own:

Condition	Question	Test
EXISTENCE	Does <code>AUDIT-NNN</code> exist at all?	<code>TestEveryCitedRuntimeRecordExists</code>
CORRECTNESS	Does the citation match the record?	<code>TestNoDocumentMisquotesTheRuntimeLedger</code>
TEMPORAL	Which state of the world is it from?	<code>TestEveryCitationCarriesATemporalStanding</code>
SEMANTIC	Is it labelled as what it is?	<code>TestEveryHistoricalCitationIsActuallyHistorical</code>

A citation of `AUDIT-099` in a fourteen-record ledger is not a content mismatch — it is a reference to nothing, and reporting it as a mismatch would send a reader hunting for a discrepancy that does not exist.

`TestTheFourConditionsAreDistinct` proves none of the four is passing because its condition is unreachable.

1.1 The SEMANTIC check found a flaw in my own marker design

On its first run it reported nine failures — all of them citations **marked historical that agree with the current artefact**.

That is a real problem: a marker suppresses checking for what it covers, so a marker that has outlived its subject is a permanently disabled check. But the rule as I first stated it was wrong. A quoted transcript legitimately contains lines identical to the current artefact — that is what makes it a transcript rather than a diff.

The marker is **block-scoped** and I had written a **line-scoped** rule. The corrected rule is stated over the block:

A marker must license at least one citation that actually differs. A marker over a block where nothing differs records nothing and disables checking, so it must be removed.

2. Temporal provenance as a type

Jangan hanya mengandalkan marker markdown.

Correct, and the reason is structural: a markdown marker can only answer the SEMANTIC question, and only by convention. `pkg/provenance/temporal` makes the answer a value.

CURRENT	agrees with the artefact as committed; may be read as a claim about now
HISTORICAL	describes a past state, quoted deliberately; makes no claim about now
SUPERSEDED	describes a past state that a NAMED successor replaced
DERIVED	computed from another reference; inherits that reference's standing
EXTERNAL	comes from outside VERIQA; the standing is the attestor's, not ours
UNVERIFIED	nobody has classified this; it may not be presented as fact

Four design decisions carry the weight:

- **The zero value is UNVERIFIED.** A reference nobody classified must not acquire standing by omission, which is exactly how the defect that prompted this package arose.
- **SUPERSEDED must name its successor.** Without one it says the same thing as HISTORICAL and the distinction collapses.
- **A contradictory reference is refused.** One marked CURRENT that also names a successor is two claims, and silently preferring one is how a stale citation survives review.
- **Promotion needs a reason; demotion does not.** Discovering something is stale should be easy to record. Promotion is how a stale claim becomes a current one.

EXTERNAL is deliberately **not** ranked above CURRENT: `TestExternalIsNotRankedAboveCurrent` fails if it ever is, because an outside attestation of a stale fact is still stale.

3. Traceability integrity — and the nine broken citations

Buktikan: Requirement → Control → Code → Test → Evidence → Report tidak bisa putus.

Every previous test checked **one hop**. The matrix checked that a control names code; another test checked that a citation resolves. Nothing checked the **chain** — and a chain is exactly the thing that can be intact at every joint and still not reach end to end, because the joints are checked by different tests that never compare notes.

`test/architecture/traceability_chain_test.go` walks all six hops. The rule is stated carefully, because an article that is honestly OPEN has no code and demanding one would push the matrix towards claiming implementations that do not exist:

The chain must be intact **up to** the point the article's own verdict says it stops, and **no further**. A link missing before that point means the verdict claims more than the chain supports. A link present after it means the verdict is stale.

3.1 What it found immediately

Nine articles cited tests that do not exist.

article 3	TestSharedUpstreamSourceIsOneSource	-> not declared anywhere
article 7	TestPolicyRetroactivityIsRefused	-> not declared anywhere
article 11	TestDissentCannotBeSuppressedAtEitherLayer	-> not declared anywhere
article 13	TestPartyCredentialsDoNotConferIndependence	-> not declared anywhere
article 14	TestUndeclaredConflictAndArticle15Honesty	-> not declared anywhere
article 20	TestPrivilegeLeakageByEveryAIRight	-> not declared anywhere
article 26	TestPolicyRetroactivityIsRefused	-> not declared anywhere
article 28	TestSharedUpstreamSourceIsOneSource	-> not declared anywhere
article 29	TestSourceOutageIsNotAFinding	-> not declared anywhere

Nine controls had a **broken TEST link** and the matrix was green regardless. Renaming a test is a refactor nobody thinks of as touching the assurance matrix, which is precisely why this link rots silently. All nine now cite tests that exist, and the check runs on every build.

The chain is also checked against the filesystem rather than against itself: `TestEveryCodeReferencePointsAtSomethingThatExists` resolves every `CodeRef` to a real package directory, so a renamed package cannot leave a control green while ceasing to exist.

4. The qualification evidence ledger

Test → Execution → Environment → Input hash → Output hash → Tool/version → Result → Evidence → Signature

`pkg/qualification/ledger` implements the assurance ladder as the review stated it:

```
IMPLEMENTED -> INTEGRATED -> ASSURED -> QUALIFIED
              -> EXTERNALLY VALIDATED -> PRODUCTION PROVEN
```

Note the ordering: **ASSURED before QUALIFIED**, and **QUALIFIED** already means *an outside party examined it* — that is what the word has meant in `pkg/assurance` since the matrix was written, and two ladders using the same word for different things would be worse than either alone. So `RequiresOutsideParty()` returns true from **QUALIFIED** up, and the internal ceiling is **ASSURED**.

What the ledger refuses:

- **A PASS with no evidence artefact.** An assertion by the party who benefits from it.
- **A PASS with no stated limitation.** Every real qualification has a boundary, and one that names none has not looked for it.
- **VERIQA as its own external validator** — `"VERIQA"`, `"ourselves"`, `"the internal team"`, `"self-assessment"` are all refused.
- **A QUALIFIED claim at the SELF_TESTED boundary.** This is the single check that stops *"we ran the test"* becoming *"it is qualified"*.

- **A REFUSED result supporting any level.** A control that declined to act was safe, and safety is not evidence of capability — the review's own point about the redaction workers, encoded.

The three boundaries the review asked to be made unmistakable:

Boundary	Means
VERIQO_SELF_TESTED	VERIQO wrote the test, ran it, reports the result
VERIQO_PROVED	holds by construction — a type constraint, a single constructor — not because a test passed; still VERIQO's own reasoning about VERIQO's own code
VERIQO_EXTERNALLY_VALIDATED	an identified party who is not VERIQO examined it

Nothing in VERIQO carries the third.

5. Article 18: the corpus architecture, and the number that matters

The review's point was the one that would be easiest to dodge:

PDF A → supported → verified. PDF B/C/D → unsupported → rejected. Ini aman. Tetapi belum membuktikan VERIQO dapat menangani real-world PDF population. Karena mungkin 60–80% dokumen real-world masuk ke B/C/D.

That is why `pkg/evidence/redaction/corpus` reports a **coverage ratio, not a pass rate**. A run where the worker refuses everything has a perfect pass rate and zero capability.

5.1 The measured result

23 structural variants: 15 accepted, 8 refused by design, 0 failed. Coverage ratio 65%.

And the number the review actually asked about:

5 refused variants are COMMON or UBIQUITOUS in real documents: PDF-ENCRYPTED (COMMON), PDF-INCREMENTAL (COMMON), PDF-OBJECT-STREAM (UBIQUITOUS), PDF-XREF-STREAM (UBIQUITOUS), PPTX-EMBEDDED-IMAGE (COMMON). A real-world population would land there in bulk.

Object streams and cross-reference streams are the dominant shape of PDF 1.5+ output. `TestTheWeightedGapIsReported` fails if PDF-OBJECT-STREAM ever stops being named explicitly, so the largest limit cannot be left for a reader to infer.

5.2 The five-level ladder, and where Article 18 actually sits

L1	static correctness	reached
L2	fixture correctness	reached — and this package is what reaches it: the variants are structurally real, and the ratio is measured rather than assumed
L3	corpus qualification	NOT reached
L4	independent	not reached
L5	production evidence	not reached

L3 is not reached and cannot be faked here. Ten thousand real PDFs cannot be synthesised, and a generated document is by construction one this codebase already understands — which is exactly the bias L3 exists to escape.

`VERIQO_CORPUS_DIR` points the same runner at a real corpus when one exists, so reaching L3 becomes a matter of supplying documents rather than writing code. `ExternalCorpusStatus()` reports:

```
L3 NOT ATTEMPTED: VERIQO_CORPUS_DIR is unset, so no corpus of documents VERIQO did not
create was available.
```

`TestL3IsNotClaimed` fails if that sentence is ever inherited rather than regenerated. Claiming L3 from generated documents would be the most comfortable lie available in this whole engagement, because the numbers would look excellent.

5.3 A worker defect the corpus found

`PDF-MALFORMED` returned **FAILED**, not **REFUSED** — the worker errored inside the cross-reference rebuild rather than declining up front. "I declined" and "I broke" are different facts about a redaction pipeline, and a corpus run that could not tell them apart would report a defect as a safe outcome. The worker now checks structural completeness before any rewriting.

6. Cross-domain semantics — six properties, and an Article 28 violation

F-6 jangan hanya Maritime → Commodity → Insurance, tetapi buktikan shared case identity, entity resolution, temporal alignment, provenance, authority, trust propagation.

Landing on one case is not the same as **being one case**. Three evidence items sharing a case id but describing three different vessels would satisfy "shared case identity" and be worthless. Each property is now proven separately, and each test states what goes wrong when it does not hold.

The one that does the real work is **entity resolution**: the three domains name the same vessel "*MV Bergensfjord*", "*M/V BERGENSFJORD*" and "*Bergensfjord*". Unless something resolves them to one entity, the case is about three vessels and its conclusion is unsound. The assertion is same-entity rather than a confidence threshold — a threshold would be a number the test could be tuned against.

6.1 The finding: UNKNOWN was being counted as INDEPENDENT

Proving **trust propagation** surfaced a real violation of Article 28.

`independence.Cluster` correctly refuses to merge on an UNKNOWN verdict — clustering answers "*which of these are the same source*", and UNKNOWN does not establish sameness. But callers were using `EffectiveSourceCount` as a **corroboration** count, and there two sources whose independence was never assessed count as two.

That is UNKNOWN read as INDEPENDENT at exactly the point where it matters. The distinction is easy to lose because the two numbers are **equal whenever everything has been assessed**, which was true in every fixture.

`EffectiveIndependentCount` is the corroboration count: a source counts only if every pairing it takes part in was assessed and found Independent, and the unassessed pairs are **returned by name** so a caller can go and assess them rather than silently losing sources.

6.2 A second finding, in the fixtures

Switching to the strict count immediately failed — and correctly. The integration fixtures populated **five** disqualifying dimensions and left three (`DataCustody`, `ModelDependency`, `SensorMeasurement`) unassessed. The cross-domain case's sources were UNKNOWN, not Independent, and only the cluster count made them look like two.

Both fixtures now derive their attributes from `independence.DisqualifyingDimensions()`, so a dimension added to the article makes them incomplete loudly instead of quietly.

7. The qualification evidence plan

`pkg/assurance/plan` covers all 28 open controls. `TestThePlanCoversEveryOpenControl` fails if one is omitted — a plan that quietly dropped a gap would let it disappear from view without being closed.

4 actionable by VERIQO today. 24 waiting on an outside party.

The headline is written so it cannot be quoted as progress:

"The second number does not shrink by working harder."

Validator	Controls
an independent assessor (none engaged)	19
VERIQO engineering	4
qualified legal counsel (none engaged)	2
an adversarial recovery lab (none engaged)	1
an external TSA (none engaged)	1
a data partner under a real data agreement (none engaged)	1

Four rules keep the plan honest, each enforced:

- `ErrSelfValidate` — VERIQO may not validate its own promotion to a level requiring an outside party.
- No **actionable** item may plan a promotion above the internal ceiling, or name an external validator.
- No blocked item may omit its blocker: *"blocked"* with no reason is indistinguishable from *"not started"*.
- `TestMostOfThePlanIsBlocked` fails if blocked items stop being the majority — which would mean either an outside party was engaged (say so) or the ladder was weakened (do not).

Article 18's validator is an **adversarial lab**, not an assessor, because its obligation is that somebody *fails to recover* redacted content. That is falsification, not examination. Its blocker states plainly that the real-world refusal rate is **unmeasured**.

No item reaches `PRODUCTION_PROVEN`. `AccreditationTrack` says why: no plan written before deployment can schedule operating under real load with real consequences.

8. Audit position

	Before	After
OPEN	2	2

	Before	After
INTEGRATION_GAP	0	0
ASSURANCE_GAP	22	22
EXTERNAL_QUALIFICATION	6	6
QUALIFIED	0	0

Nothing moved, and nothing should have. This round added no capability to any article. It added the machinery to say what would legitimately move them, and it repaired nine broken traceability links and one Article 28 violation that the previous position had been resting on.

A round that improved the numbers while doing this work would have been the suspicious outcome.

9. Verification

`scripts/verify.sh`, all six stages, exit 0. The new packages use only the standard library, so the zero-external-dependency invariant holds.

New this round:

- `pkg/provenance/temporal` — the six-state vocabulary
- `pkg/qualification/ledger` — the assurance ladder and evidence ledger
- `pkg/assurance/plan` — the 28-item qualification evidence plan
- `pkg/evidence/redaction/corpus` — 23 structural variants, the coverage ratio, the external-corpus harness
- `test/architecture/traceability_chain_test.go` — the six-hop chain
- `test/integration/cross_domain_semantics_test.go` — the six properties

9.1 Defects found this round

Three by the new checks, two in my own work:

- **Nine articles cited tests that do not exist** (§3.1).
 - **UNKNOWN counted towards corroboration**, an Article 28 violation in the trust layer (§6.1).
 - **Integration fixtures left three disqualifying dimensions unassessed** (§6.2).
 - **PDF-MALFORMED failed rather than refusing** (§5.3).
 - **My SEMANTIC rule was line-scoped where the marker is block-scoped** (§1.1).
 - The `gofmt -reflow` trap caught a string replacement again — the fourth time in this engagement. Caught by grepping for the result.
-

10. What this round does not claim

- Article 18 is at the **L2/L3 boundary**, exactly as the review placed it. 65% coverage over variants VERIQO generated; the real-world population is unmeasured and expected to be worse.
- The cross-domain case remains fixtures. The six semantic properties hold **on those fixtures**.
- The qualification plan is a plan. Nothing in it has been executed by an outside party, because no outside party is engaged.

- The scanner rules key on return type and construction rather than on function names — so renaming `Resolve` to `Compute` does not defeat them. Full data-flow analysis is not implemented, and a determined developer working around a structural rule is not something a structural rule catches.
- Nothing is QUALIFIED. Nothing is above the internal ceiling.

The next boundary has not changed and is not another architectural round: real rights-aware data under a real agreement, an adversarial lab for Article 18, and an assessor who is not VERIQO.